Module -5 (Other Models)

Module - 5 (Other Models) Advanced models: Booch's Methodology- Notations, models, concepts. Jacobson Methodology architecture, actors and use-cases, requirement model, Analysis Model, Design model, Implementation model and Test Model-Unified Modeling Language (UML).

- The Booch Method
  - The Booch Method is a well-documented, proven method for object-oriented analysis and design.
  - It is composed of an object modeling language, an iterative object-oriented development process, and a set of recommended practices
  - This model was proposed by Grady Booch in 1992 when he was working for Rational Software which was later acquired by IBM.
  - It was widely used in software engineering for object-oriented analysis and design and benefited from ample documentation and support tools.

What is Booch's Object Identification Method?

- Booch's Object identification model was used to identify the different objects of software while doing domain analysis (domain modeling).
- This model was proposed by Grady Booch in 1992 when he was working for Rational Software which was later acquired by IBM.
- This model was then again revised by him in 1994.
- This model was widely used in software engineering for data modeling and object-oriented domain analysis purpose

- Booch's method was taken from the concepts of UML (Unified Modelling Langauge).
- It is widely used method object oriented method to design a system.
- It helps to design our system using object paradigm.
- It covers the analysis and design of an object oriented systems.
- Booch uses large set of symbols.

- What makes the Booch Method different?
  - The Booch method notation differs from other OO methodologies because it centers on the development of four fundamental models of the system in the problem domain that will be implemented primarily in software.
  - These models are:
    - The Logical Model
    - The Physical Model
    - The Static Model
    - The Dynamic Model

- These models document components: classes, class categories, objects, operations,subsystems, modules, processes, processors, devices and the relationships between them.
- Just as an architect has multiple views of a skyscraper under construction, so the software engineer has multiple view of the software system undergoing design.

Views of the Booch Method models

- The Booch method recognizes that an engineer can view the four models from several different perspectives. In the Booch method, an engineer can document the models with several different diagrams:

1. The Class Diagram
2. The Object Diagram
3. The Module Diagram
4. The State Diagram
5. The Interaction Diagram
6. The Process Diagram

- The engineer augments these diagrams with specifications that textually complement the icons in the diagrams.
- It's important to realize that there is not a one-to-one mapping between a model and a particular view.
- Some diagrams and/or specifications can contain information from several models, and some models can include information that ends up in several diagrams.
- Each model component and their relationships can appear in none,one, or several of a models diagram.

i) Class Diagram

- A graphical representation of the static view of the system and represent different aspects of the application.
- A collection of class diagrams represent the whole system.
- Show the existence of classes and their relationships in the logical view of the system.
- The class diagram is the main building block of object-oriented modeling.
- It is used for general conceptual modeling of the structure of the application, and for detailed modeling, translating the models into programming code.
- Class diagrams can also be used for data modeling.

Class Diagram - RELATIONSHIPS

Inheritance

Association

Aggregation

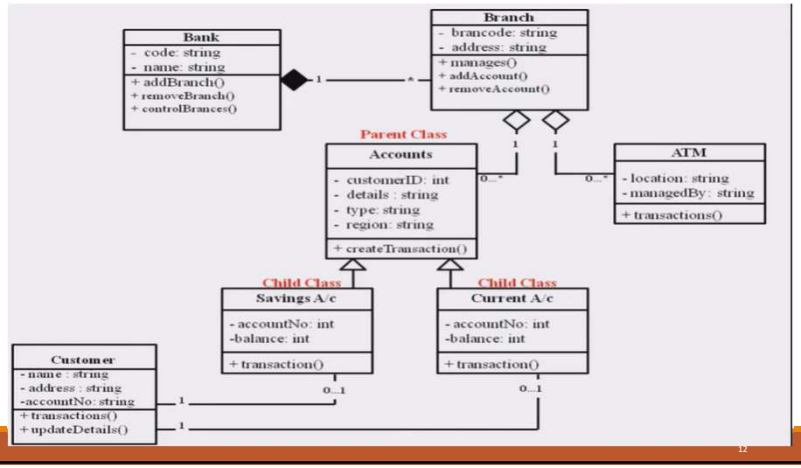
Composition

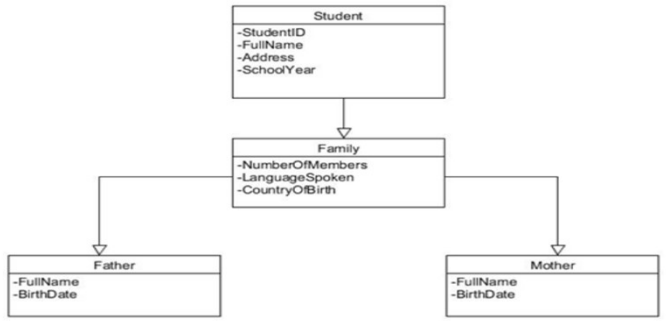
Visibility

- – indicates a private member
- + indicates a public member
- # indicates a protected member.

Multiplicity

- 0..1 zero to one
- n specific number
- 0..* zero to many
- 1..* one to many
- m..n specific number range



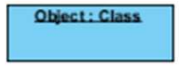


Object Diagram

- Shows the existence of objects , their relationships in the logical view of the system.
- Basic Object Diagram Symbols and Notations

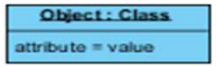
1. Object Names:

Every object is actually symbolized like a rectangle, that offers the name from the object and its class underlined as well as divided with a colon.



2. Object Attributes:

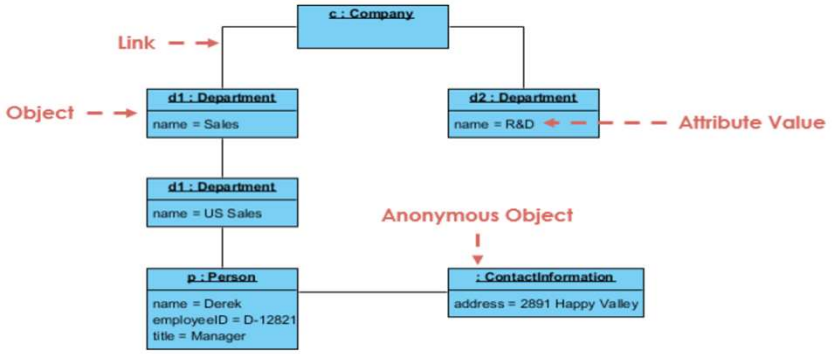
Similar to classes, you are able to list object attributes inside a separate compartment.



3. Links:

Links tend to be instances associated with associations. You can draw a link while using the lines utilized in class diagrams.

Object Diagram Example I - Company Structure



State Diagram

1. **Initial state** – We use a black filled circle represent the initial state of a System or a class.

Figure – initial state notation

2. **Transition** – We use a solid arrow to represent the transition or change of control from one state to another. The arrow is labelled with the event which causes the change in state.

Figure – transition

3. **State** – We use a rounded rectangle to represent a state. A state represents the conditions or circumstances of an object of a class at an instant of time.

4. **Fork** – We use a rounded solid rectangular bar to represent a Fork notation with incoming arrow from the parent state and outgoing arrows towards the newly created states. We use the fork notation to represent a state splitting into two or more concurrent states.

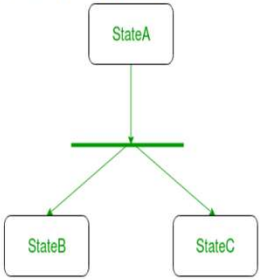


Figure – a diagram using the fork notation

5. **Join** – We use a rounded solid rectangular bar to represent a Join notation with incoming arrows from the joining states and outgoing arrow towards the common goal state. We use the join notation when two or more states concurrently converge into one on the occurrence of an event or events.

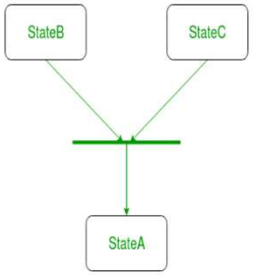


Figure – a diagram using join notation

6. **Self transition** – We use a solid arrow pointing back to the state itself to represent a self transition. There might be scenarios when the state of the object does not change upon the occurrence of an event. We use self transitions to represent such cases.

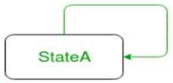


Figure – self transition notation

7. **Composite state** – We use a rounded rectangle to represent a composite state also.We represent a state with internal activities using a composite state.

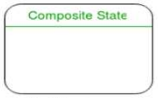


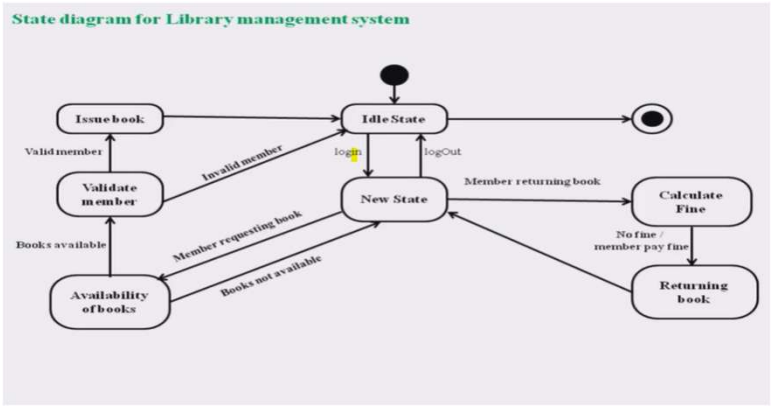
Figure – a state with internal activities

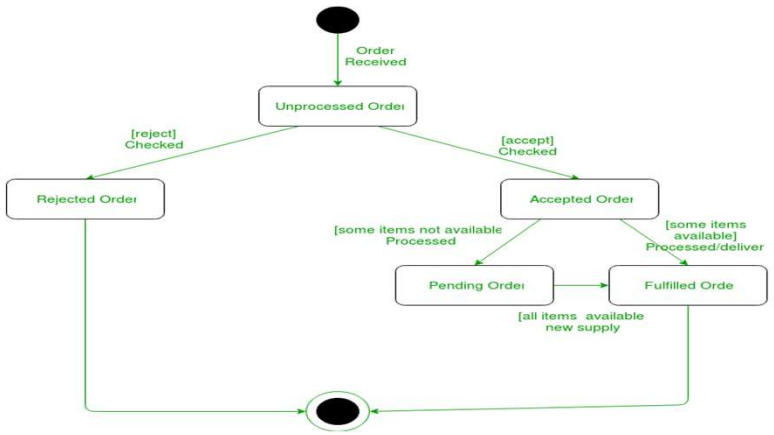
8. **Final state** – We use a filled circle within a circle notation to represent the final state in a state machine diagram.

Figure – final state notation

Steps to draw a state diagram –

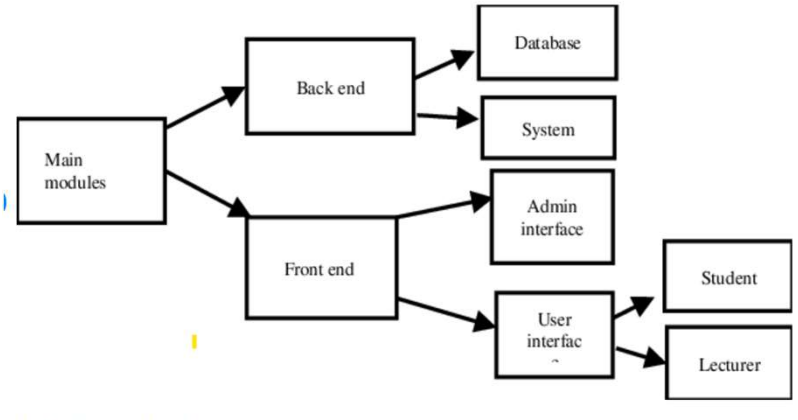
1. Identify the initial state and the final terminating states.
2. Identify the possible states in which the object can exist (boundary values corresponding to different attributes guide us in identifying different states).
3. Label the events which trigger these transitions.





Module Diagram

- A module diagram is a type of diagram that represents the components, modules, and their relationships in a system.
- Modules are typically used to group related functionality, and can contain classes, interfaces, or other modules.
- Shows the allocation of classes and objects to modules in the physical view of a system



Module Diagram of the System

Interaction Diagrams

- The interaction diagrams illustrate how objects interact via messages.
- They are used for dynamic object modeling.

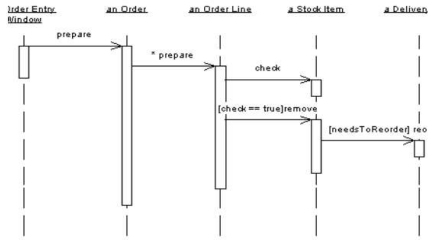


Figure: An interaction diagram

Process Diagram

- It is used to show allocation of processes to processors in the physical design.
- The following shows the notation for processes and devices.

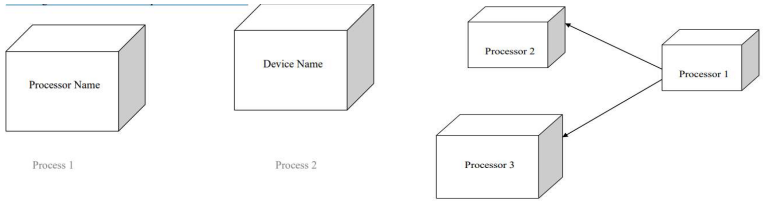


Figure: An example process diagram

- The Booch's method prescribes a macro development process and a micro development process:
- a) Macro Development Process
- The macro process serves as a controlling framework for the micro process and can take weeks or even months.
- The primary concern of the macro process is technical management of the system .
- Such management is interested less in the actual OOD than in how well the project corresponds to the requirements set for it and whether it is produced on time.

- The macro development process consists of the following steps:
- i) Conceptualization:
  - Establish the core requirements of the system .
  - Establish a set of goals and develop a prototype to prove the concept.
- ii) Analysis and Development of the Model:
  - Class diagrams are used to describe the roles and responsibilities objects are to carry out in performing the desired behavior of the system.
  - The object diagrams describe the desired behavior of the system in terms of scenario.

iii) Design or create the system architecture:

- The class diagram is used to decide what classes exist and how they relate to each other.
- The object diagrams decide what mechanisms are used to regulate how objects collaborate .
- The module diagram is used to map out where each class and object should be declared.
- The process diagram to determine to which processor .
- Also determine the schedules for multiple processes on each relevant processor

- iv) Evolution or implementation:
  - Successively refine the system through many iterations .
  - Produce a stream of software implementations, each of which is a refinement of the prior one.
- v) Maintenance:
  - Making localized changes to the system to add new requirements and eliminate bugs.

- b) Micro Development Process:
  - Each macro development process has its own micro development processes.
  - It is a description of the day-to-day activities by a single or small group of software developers, which looks blurry to an outsider, since analysis and design phases are not clearly defined.
  - It consists of the following:

i) Identify classes and objects

ii) Identify class and object semantics

iii) Identify class and object relationships

iv) Identify class and object interfaces and implementation

The Jacobson's Methodology

- The Jacobson's methodologies (Object oriented Business Engineering (OOBE),Object Oriented Software Engineering (OOSE) and Objectory) cover the entire life cycle and stress traceability between the different phases, both forward and backward.
- This traceability enables reuse of analysis and design work, possibly much bigger factors in the reduction of development time than reuse of code. At the heart of their methodologies is the use-case concept, which evolved with Objectory.

i) Use cases

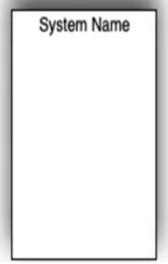
- Use cases are scenarios for understanding system requirements .
- A use case is an interaction between users and a system.
- The use case model captures the goal of the user and the responsibility of the system to its users.
- In the requirement analysis, the use case is described as one of the following:
  - Non-formal text with no clear flow of events.
  - Text, easy to read but with a clear flow of events to follow (this is a recommended style).
  - Formal style using pseudocode.

- The use case description must contain:
  - when How and when the use case begins and ends.
  - The interaction between the use case and its actors, including when the interaction occurs and what is exchanged.
  - How and when the use case will need data stored in the system or will store data in the system.
  - Exceptions to the flow of events.
  - How and concepts of the problem domain are handled

- Every single use case should describe one main flow of events.
- An exceptional or additional flow of events could be added.
- The exceptional or additional flow of events could be added.
- The exceptional use case extends another use case to include the additional one.
- The use case model employs extends and uses relationships.
- The extends relationship is used when you have one use case that is similar to
- another use case but does a bit more .
- It extends the functionality of the original use case

- The uses relationship senses common behavior in different use cases.
- Use cases could be viewed as concrete or abstract.
- An abstract use case is not complete and has no actors that initiate it but is used by another use case .
- This inheritance could be in several levels.
- Abstract use cases are the ones that have uses or extend relationships

1. System The project that you are developing is a system.



application software, a website, software component, business process, or any other projects

2. Actor

Anyone how use to perform a certain function in the system is actors.

customer, staff, admin, bank, student, teacher, etc.

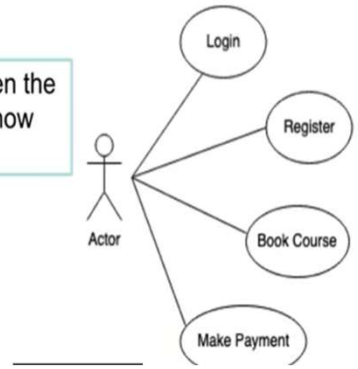
3. Use Cases

All the functionality that the system does is the USE CASE.

login, check balance, transfer fund, make a payment

4. Relationship

we draw the solid line between the Actor and the Use Case to show the relationship.



Type of Relationship in addition to the Association

Include

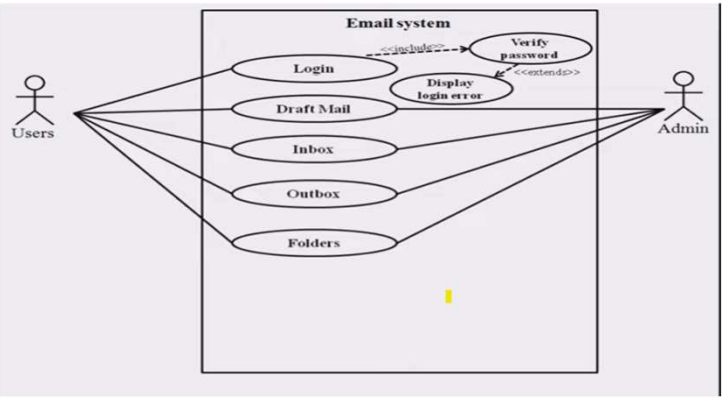
Extend

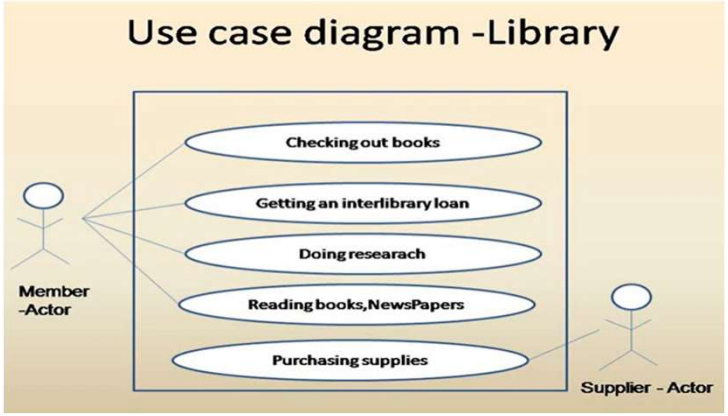
Include relationship

- An include relationship shows dependency between a base case and an included use case.
- When a use case is depicted as using the functionality of another use case, the relationship between the use cases is named as include or uses relationship.

Extend relationships

- The extend relationships are important because they show optional functionality or system behaviour.
- The <<extend>> relationship is used to include optional behaviour from an extending use case in an extended use case.



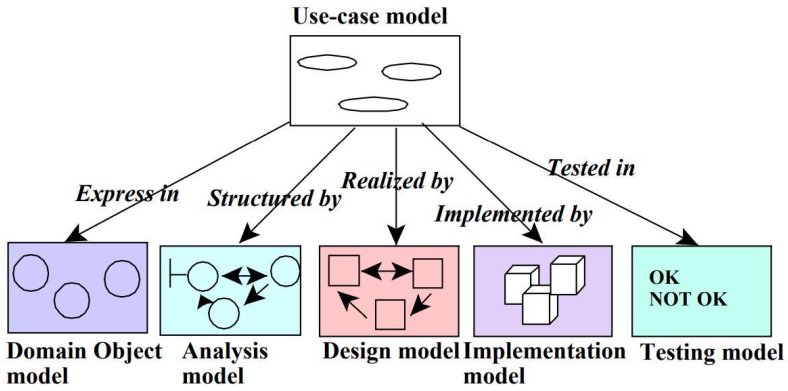


Purpose of Use Case Diagram

- Use case diagrams are typically developed in the early stage of development and people often apply use case modelling for the following purposes:
  - Specify the context of a system
  - Capture the requirements of a system
  - Validate a systems architecture
  - Drive implementation and generate test cases
  - Developed by analysts together with domain experts

- ii) Architecture
- System development includes development of different models. We need to find modeling technique notation for each model.
- Such a set of modeling techniques defines the architecture.
- Each model shows certain aspects of the system.
- Collection of models is known as architecture.

- iii) Object Oriented Software Engineering:
- It is also called as Objectory.
- It is a method of object oriented development with the specific aim to fit the development of large, real time systems.
- The development process also use case driven development , stresses that use cases are involved in several phases of the development , including analysis , design ,validation and testing.
- The use case scenario begins with a user of the system initiating a sequence of interrelated events.
- The system development method based on OOSE, Objectory is a disciplined process for the industrialized development of software based on a use case driven design.



- It is an approach to OOAD that centers on understanding the ways in which a system actually is used.
- By organizing the analysis and design models around sequences of user interaction and actual usage scenarios, the method produces systems that are both usable and more robust adapting more easily to changing usage.
- Objectory has been developed and applied to numerous application areas and embodied in CASE tool systems.

- Objectory is built around several different models:
- Use Case Model:
  - The use case model defines the outside (actors) and inside (use case) of the system' behavior.
- Domain Object Model:
  - The objects of the real world are mapped into the domain object model.
- Analysis object model:
  - The analysis model presents how the source code (implementation) should be carried out and written.
- Implementation Model:
  - The implementation model represents the implementation of the system.
- Test Model:
  - The test model constitutes the test plans, specifications and reports.

- The maintenance of each model is specified in its associated process.
- A process is created when the first development project starts and is terminated when the developed system is taken out of service.

iv) Object Oriented Business Engineering:

- OOBE is object modeling at the enterprise level .
- Use cases again are the central vehicle for modeling, providing traceability throughout the software engineering processes. OOBE consists of:

1. Analysis phase
2. Design
3. Implementation phases
4. Testing phase.

- Analysis Phase:
- It defines the system to be built in terms of the problem domain object model, the requirement model and analysis model.
- It shouldn't take into account the actual implementation model.
- It reduces complexity and promotes maintainability over system.
- Jacobson doesn't insist on development of problem domain object model, but refers to other models, who suggest that customers draw his view of system.
- This model should be developed just understanding for requirements model.
- The analysis process is iterative but requirements and analysis model should be stable before moving on to subsequent models.
- Prototyping is also useful

Design and Implementation Phases:

- Implementation environment should be identified for design model.
- The various factors affecting the design model are DBMS, distribution of process, constraints due to programming language, available component libraries and incorporation of GUI.
- It may be possible to identify implementation environment concurrently with analysis.
- The analysis objects are translated into design objects that fit current implementation environment.

- Testing Phase:
- It describes testing, levels and techniques (unit, integration and system.)

- **UML**
- UML stands for Unified Modeling Language.
- This object-oriented system of notation has evolved from the work of Grady Booch, James Rumbaugh, Ivar Jacobson, and the Rational Software Corporation.
- These renowned computer scientists fused their respective technologies into a single, standardized model.
- Today, UML is accepted by the Object Management Group (OMG) as the standard for modeling object oriented programs.
- It is an industry-standard graphical language for specifying, visualizing, constructing, and documenting the artefacts of software systems.
- The UML uses mostly graphical notations to express the object oriented analysis and design of software projects.
- Simplifies the complex process of software design.

- The primary goals in the design of the UML were as follows:

1. Provide users a ready to use expressive visual modeling language so they can develop and exchange meaningful models.
2. Provide extensibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development processes.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of the OO tools market.
6. Support higher-level development concepts.
7. Integrate best practices and methodologies

- **Types of UML Diagrams**
- UML defines nine types of diagrams:
- Class (package)
- Object,
- Use case,
- Sequence,
- Collaboration,
- State chart,
- Activity,
- Component,
- Deployment.

•**i) UML Class Diagram**

•Class diagrams are the backbone of almost every object-oriented method including UML.

•They describe the static structure of a system.

•**Basic Class Diagram Symbols and Notations:**

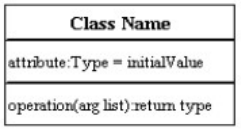
•Classes represent an abstraction of entities with common characteristics.

• Associations represent the relationships between classes.

•**Classes**

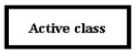
•Illustrate classes with rectangles divided into compartments.

•Place the name of the class in the first partition (centered, bolded, and capitalized), list the attributes in the second partition, and write operations into the third.



Active Class

- Active classes initiate and control the flow of activity, while passive classes store data and serve other classes.
- Illustrate active classes with a thicker border.



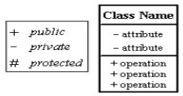
•**Visibility**

•Use visibility markers to signify who can access the information contained within a class.

• Private visibility hides information from anything outside the class partition.

•Public visibility allows all other classes to view the marked information

• Protected visibility allows child classes to access information they inherited from a parent class



Associations

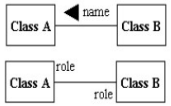
• Associations represent static relationships between classes.

•Place association names above, on, or below the association line.

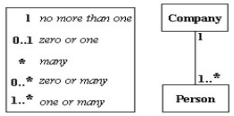
• Use a filled arrow to indicate the direction of the relationship.

•Place roles near the end of an association.

•Roles represent the way the two classes see each other.

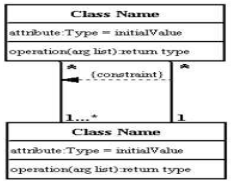


- **More Basic Class Diagram Symbols and Notations**
- Multiplicity(Cardinality)
- Place multiplicity notations near the ends of an association.
- These symbols indicate the number of instances of one class linked to one instance of the other class.
- Eg:one company will have one or more employees, but each employee works for one company only.

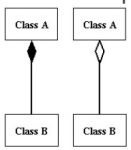


- **Constraint**
- Place constraints inside curly braces {}.

Simple Constraint

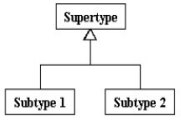


- **Composition and Aggregation**
- Composition is a special type of aggregation that denotes a strong ownership between Class A, the whole, and Class B, its part.
- Illustrate composition with a filled diamond.
- Use a hollow diamond to represent a simple aggregation relationship, in which the "whole" class plays a more important role than the "part" class, but the two classes are not dependent on each other.
- The diamond end in both a composition and aggregation relationship points toward the "whole" class or the aggregate.



Generalization

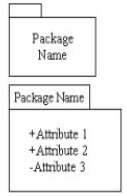
- Generalization is another name for inheritance or an "is a" relationship.
- It refers to a relationship between two classes where one class is a specialized version of another.
- Eg: Honda is a type of car.
- So the class Honda would have a generalization relationship with the class car



- In real life coding examples, the difference between inheritance and aggregation can be confusing.
- If you have an aggregation relationship, the aggregate (the whole) can access only the PUBLIC functions of the part class.
- On the other hand, inheritance allows the inheriting class to access both the PUBLIC and PROTECTED functions of the superclass.

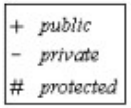
- ii) UML Package Diagram
- Package diagrams organize the elements of a system into related groups to minimize dependencies among them

- **Basic Package Diagram Symbols and Notations**
- Packages
- Use a tabbed folder to illustrate packages.
- Write the name of the package on the tab or inside the folder.
- Similar to classes, you can also list the attributes of a package.



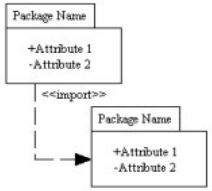
Visibility

- Visibility markers signify who can access the information contained within a package.
- Private visibility means that the attribute or the operation is not accessible to anything outside the package.
- Public visibility allows an attribute or an operation to be viewed by other packages.
- Protected visibility makes an attribute or operation visible to packages that inherit it only.

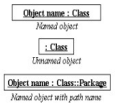


Dependency

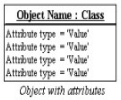
- Dependency defines a relationship in which changes to one package will affect another package.
- Importing is a type of dependency that grants one package access to the contents of another package.



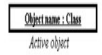
- iii) UML Object Diagram
- Object diagrams are also closely linked to class diagrams.
- An object diagram could be viewed as an instance of a class diagram.
- Object diagrams describe the static structure of a system at a particular time and they are used to test the accuracy of class diagrams.
- **Basic Object Diagram Symbols and Notations**
- Object names
- Each object is represented as a rectangle, which contains the name of the object and its class underlined and separated by a colon



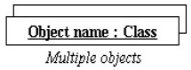
- Object attributes
- As with classes, you can list object attributes in a separate compartment.
- Object attributes must have values assigned to them



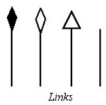
- Active object
- Objects that control action flow are called active objects.
- Illustrate these objects with a thicker border



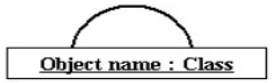
- Multiplicity
- You can illustrate multiple objects as one symbol if the attributes of the individual objects are not important.



- Links
- Links are instances of associations.
- You can draw a link using the lines used in class diagrams.

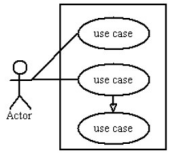


- Self-linked
- Objects that fulfill more than one role can be self-linked.
- Eg: if Mark, an administrative assistant, also fulfilled the role of a marketing assistant, and the two positions are linked, Mark's instance of the two classes will be self-linked



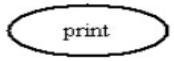
- iv) UML Use Case Diagram
- Use case diagrams model the functionality of a system using actors and use cases
- Use cases are services or functions provided by the system to its users.

- **Basic Use Case Diagram Symbols and Notations**
- System
- Draw your system's boundaries using a rectangle that contains use cases. Place actors outside the system's boundaries.



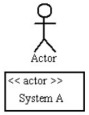
77

- Use Case
- Draw use cases using ovals.
- Label with ovals with verbs that represent the system's functions



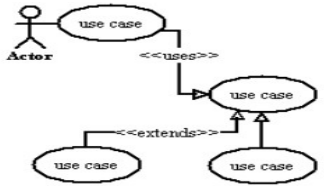
78

- Actors
- Actors are the users of a system.
- When one system is the actor of another system, label the actor system with the actor stereotype



79

- Relationships
- Illustrate relationships between an actor and a use case with a simple line.
- For relationships among use cases, use arrows labeled either "uses" or "extends."
- A "uses" relationship indicates that one use case is needed by another in order to perform a task.
- An "extends" relationship indicates alternative options under a certain use case.

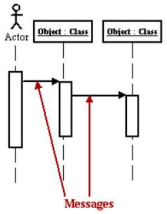


80

- v) UML Sequence Diagram
- Sequence diagrams describe interactions among classes in terms of an exchange of messages over time

- Basic Sequence Diagram Symbols and Notations
- Class roles Class roles describe the way an object will behave in context.
- Use the UML object symbol to illustrate class roles, but don't list object attributes

- Activation
- Activation boxes represent the time an object needs to complete a task



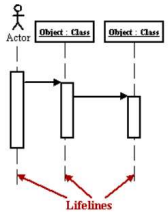
Messages

- Messages are arrows that represent communication between objects.
- Use half-arrowed lines to represent asynchronous messages.
- Asynchronous messages are sent from an object that will not wait for a response from the receiver before continuing its tasks.

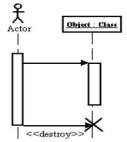
Arrow	Message type
→	Simple
→	Synchronous
→	Asynchronous
↩	Balking
◷→	Time out

Various message types for Sequence and Collaboration diagrams

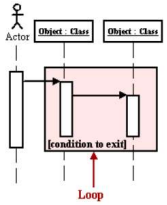
- Lifelines
- Lifelines are vertical dashed lines that indicate the object's presence over time.



- Destroying Objects
- Objects can be terminated early using an arrow labeled "< < destroy > >" that points to an X

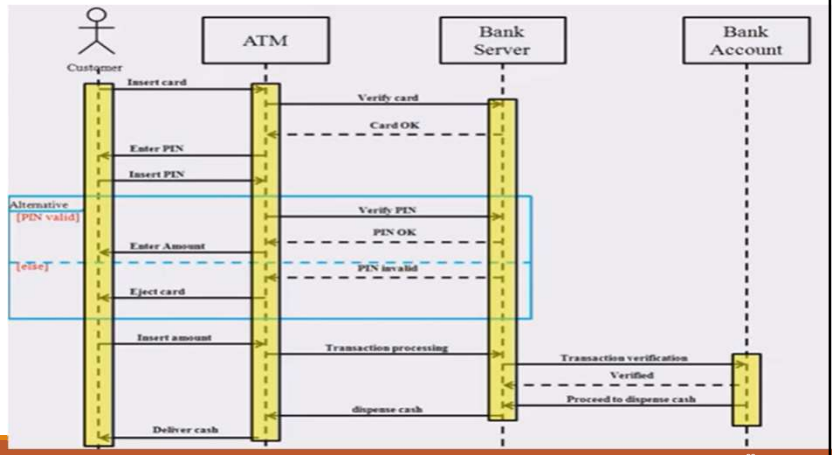


- Loops
- A repetition or loop within a sequence diagram is depicted as a rectangle.
- Place the condition for exiting the loop at the bottom left corner in square brackets [ ].

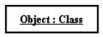


Sequence Diagram for ATM transaction

- User
- ATM
- Bank server
- Bank account

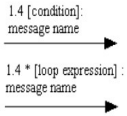


- vi) UML Collaboration Diagram
- A collaboration diagram describes interactions among objects in terms of sequenced messages.
- Collaboration diagrams represent a combination of information taken from class, sequence, and use case diagrams describing both the static structure and dynamic behavior of a system.
- **Basic Collaboration Diagram Symbols and Notations**
- Class roles
- Class roles describe how objects behave.
- Use the UML object symbol to illustrate class roles, but don't list object attributes.



- Association roles
- Association roles describe how an association will behave given a particular situation.
- You can draw association roles using simple lines labeled with stereotypes.

- Messages
- Unlike sequence diagrams, collaboration diagrams do not have an explicit way to denote time and instead number messages in order of execution.
- Sequence numbering can become nested using the Dewey decimal system.
- For example, nested messages under the first message are labeled 1.1, 1.2, 1.3, and so on.
- The condition for a message is usually placed in square brackets immediately following the sequence number.
- Use a * after the sequence number to indicate a loop



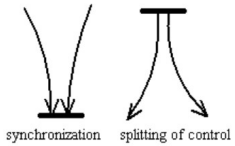
- vii) UML State chart
- Diagram A state chart diagram shows the behavior of classes in response to external stimuli.
- This diagram models the dynamic flow of control from state to state within a system.

- **Basic State chart Diagram Symbols and Notations**
- **States**
- States represent situations during the life of an object.
- You can easily illustrate a state in SmartDraw by using a rectangle with rounded corners.
- **Transition**
- A solid arrow represents the path between different states of an object.
- Label the transition with the event that triggered it and the action that results from it

- **More Basic Collaboration Diagram Symbols and Notations**
- Initial State
- A filled circle followed by an arrow represents the object's initial state.

- Final State
- An arrow pointing to a filled circle nested inside another circle represents the object's final state.

- **Synchronization and Splitting of Control**
- A short heavy bar with two transitions entering it represents a synchronization of control.
- A short heavy bar with two transitions leaving it represents a splitting of control that creates multiple states.



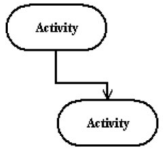
97

- viii) UML Activity Diagram
- An activity diagram illustrates the dynamic nature of a system by modeling the flow of control from activity to activity.
- An activity represents an operation on some class in the system that results in a change in the state of the system.
- Activity diagrams are used to model workflow or business processes and internal operation.
- Because an activity diagram is a special kind of state chart diagram, it uses some of the same modeling conventions.

98

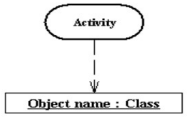
- **Basic Activity Diagram Symbols and Notations**
- Action states
- Action states represent the non interruptible actions of objects.
- You can draw an action state in SmartDraw using a rectangle with rounded corners.

- Action Flow
- Action flow arrows illustrate the relationships among action states



99

- **Object Flow**
- Object flow refers to the creation and modification of objects by activities.
- An object flow arrow from an action to an object means that the action creates or influences the object.
- An object flow arrow from an object to an action indicates that the action state uses the object.

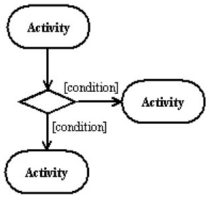


100

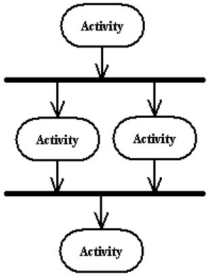
- **More Basic Collaboration Diagram Symbols and Notations**
- **Initial State**
- A filled circle followed by an arrow represents the initial action state.

- **Final State**
- An arrow pointing to a filled circle nested inside another circle represents the final action state.

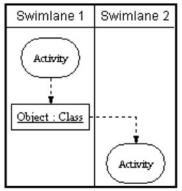
- **Branching**
- A diamond represents a decision with alternate paths.
- The outgoing alternates should be labeled with a condition or guard expression.
- You can also label one of the paths "else."

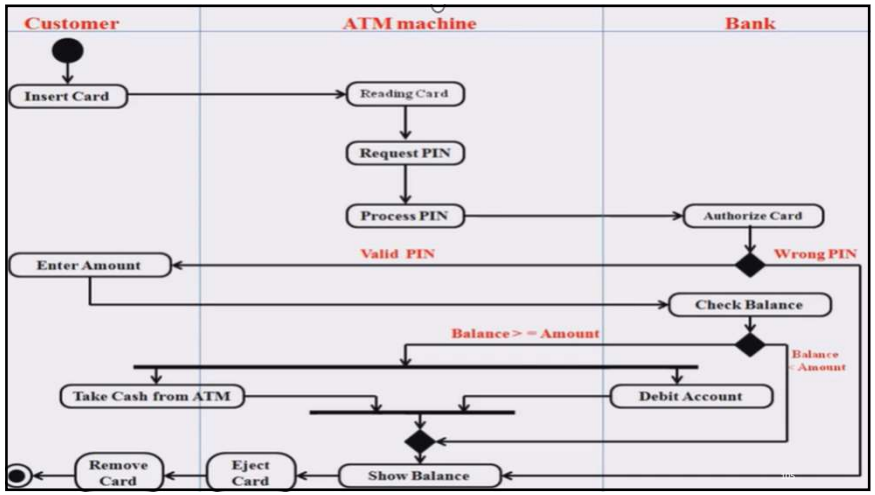


- **Synchronization**
- A synchronization bar helps illustrates parallel transitions.
- Synchronization is also called forking and joining



- Swimlanes
- Swimlanes group related activities into one column.





Component diagram

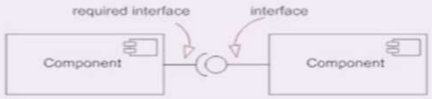
- A component diagram describes the organization and wiring of the physical components in a system.
- Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development.

Component

- A component is a logical unit block of the system, a slightly higher abstraction than classes.
- It is represented as a rectangle with a smaller rectangle in the upper right corner with tabs or the word written above the name of the component to help distinguish it from a class.

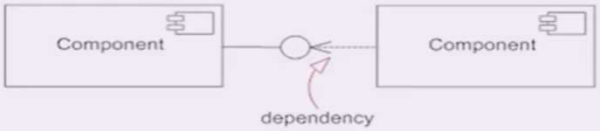
Interface

- An interface describes a group of operations used (required) or created (provided) by components.
- A full circle represents an interface created or provided by the component.
- A semi-circle represents a required interface, like a person's input.



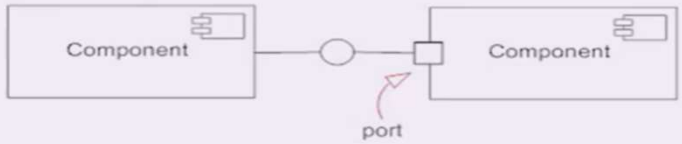
Dependencies

- Draw dependencies among components using dashed arrows.

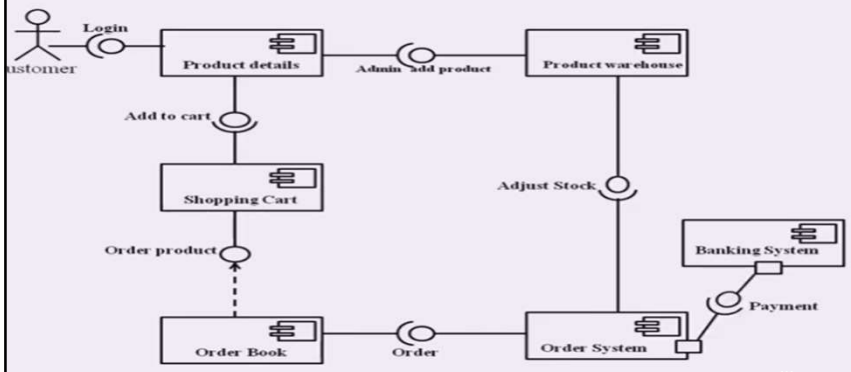


Port

- Ports are represented using a square along the edge of the system or a component. A port is often used to help expose required and provided interfaces of a component.



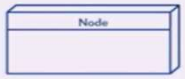
Online Shopping



Deployment diagram

- A deployment diagram is a UML diagram type that shows the execution architecture of a system, including nodes such as hardware or software execution environments, and the middleware connecting them.
- Deployment diagrams are typically used to visualize the physical hardware and software of a system. Using it you can understand how the system will be physically deployed on the hardware.
- Deployment diagrams help model the hardware topology of a system compared to other UML diagram types which mostly outline the logical components of a system.

- A **node**, represented as a cube, is a **physical entity** that executes one or more components, subsystems or executables. A node could be a **hardware or software** element.



- **Artifacts** are concrete **elements that are caused by a development process.** Examples of artifacts are libraries, archives, configuration files, executable files etc.

- **Association** is represented by a solid line between two nodes. It shows the path of communication between nodes.